In [ ]:
```python
from abc import ABC, abstractmethod

class Vehiculo(ABC):
    """Clase abstracta Vehiculo para poder crear subclases de tipos de vehículo
    def __init__(self, placa : str, kilometraje : float, costo_base_mantenimient
        self.placa = placa
        self.kilometraje = kilometraje
        self.costo_base_mantenimiento = costo_base_mantenimiento

    """Método abstracto para calcular el impuesto de las subclases de tipos de v
    @abstractmethod
    def calcular_impuesto(self) -> float:
        return 0.0

    """Método que permite actualizar el kilometraje de cualquier tipo de vehícul
    def actualizar_kilometraje(self, nuevos_km : float) -> float:
        if nuevos_km <= self.kilometraje:
            print(f"Ingresa una cantidad de kilometros mayor a la actual: {self.
            return
        self.kilometraje = nuevos_km
        print(f"Se ha actualizado el kilometraje a: {self.kilometraje}")
        return

class Auto(Vehiculo):
    def __init__(self, placa : str, kilometraje : float, costo_base_mantenimient
        super().__init__(placa, kilometraje, costo_base_mantenimiento)
        self.es_electrico = es_electrico

    def calcular_impuesto(self) -> float:
        """Método heredado de la clase abstracta que retorna el impuesto de los
        if self.es_electrico:
            return 0.0
        return 0.05 * self.kilometraje

class Camion(Vehiculo):
    def __init__(self, placa : str, kilometraje : float, costo_base_mantenimient
        super().__init__(placa, kilometraje, costo_base_mantenimiento)
        self.ejes = ejes

    def calcular_impuesto(self) -> float:
        """Método heredado de la clase abstracta que retorna el impuesto de los
        return (0.10 * self.kilometraje) + (50.0 * self.ejes)
```

Experimento de validación

In [ ]:
```python
Vehiculo("1", 0, 10)

"""---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 Vehiculo("1", 0, 10)

TypeError: Can't instantiate abstract class Vehiculo without an implementation f
```

In [ ]:
```python
class Moto(Vehiculo):
    def __init__(self, placa : str, kilometraje : float, costo_base_mantenimient
```

```
        super().__init__(placa, kilometraje, costo_base_mantenimiento)
        self.cilindrada = cilindrada
```

In [ ]:
```
Moto("2", 0.0, 100.0, 200.0)

"""TypeError                                 Traceback (most recent call last)
Cell In[7], line 1
----> 1 Moto("2", 0.0, 100.0, 200.0)

TypeError: Can't instantiate abstract class Moto without an implementation for a
```